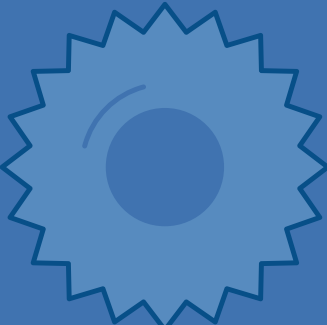




INTRODUCTION



ATM Interface System with ML-Based Fraud Detection

- **NAME:** Bhoomika payasi
- **REG.NO:** 25MIM10036
- **PROGRAM:** Int Mtech (AIML)
- **ACADAMIC YEAR:** Fall semester2026-27
- **COURSE:** Fundamentals in AIML.
- **Domain:** Software Development + Artificial Intelligence/Machine Learning

PROBLEM STATEMENT

Traditional ATM simulation projects validate transactions using only fixed, hardcoded rules — for example, blocking a withdrawal simply because it exceeds a set limit. This approach does not reflect how real banking systems detect fraud, where suspicious activity is identified by learning patterns in data rather than relying on static thresholds.

There is a need for an ATM system that:

- Performs core banking operations reliably (secure login, deposit, withdrawal, balance check)
- Goes beyond fixed rules by learning what "normal" transaction behavior looks like
- Automatically flags transactions that deviate from typical patterns — such as unusually large amounts or transactions at odd hours — for review

Proposed Solution:

- Build a modular ATM interface in Python covering all standard operations
- Integrate an unsupervised ML model (Isolation Forest) that analyzes transaction amount, time of day, day of week, and balance ratio
- Use the model to flag statistically unusual transactions in real time, bringing the system closer to real-world banking fraud detection

Functional Requirements

Balance Inquiry Module

- Display account holder name, account number, and current balance
- Include a timestamp using the datetime module

Fraud Alert Module

- Log every flagged transaction with timestamp, type, and amount
- Provide a menu option to view all logged fraud alerts

Menu System

- Present a continuous, loop-based menu until the user exits
- Handle invalid menu choices gracefully

Login Module

- Accept and validate user PIN
- Allow a maximum of 3 attempts
- Temporarily block the card after repeated failed attempts

Withdrawal Module

- Validate sufficient balance before processing
- Reject invalid (zero/negative) withdrawal amounts
- Run every withdrawal through the ML fraud detection check
- Prompt user confirmation if a withdrawal is flagged as unusual

Deposit Module

- Accept only valid, positive deposit amounts
- Reject zero, negative, or non-numeric input
- Update and display the new balance instantly
- Run every deposit through the ML fraud detection check

Non-functional Requirements

- **Usability** — simple, intuitive, menu-driven console interface requiring no technical knowledge to operate
- **Reliability** — consistent behavior across repeated use; proper handling of invalid inputs and edge cases
- **Performance** — ML fraud scoring happens instantly (milliseconds), causing no noticeable delay to the user
- **Maintainability** — modular file structure allows each feature to be updated or debugged independently
- **Portability** — runs on any system with Python 3 and the required libraries installed; no OS-specific dependencies
- **Scalability** — the ML model architecture supports retraining on larger, real-world datasets without redesigning the system
- **Security** — PIN-based access control combined with anomaly detection strengthens transaction security
- **Auditability** — all flagged transactions are logged persistently for later review

System Architecture

Three-tier architecture:

- **Presentation Layer** — `main.py`; handles the console menu and user interaction
- **Business Logic Layer** — `login_page.py`, `deposit_money.py`, `withdraw_money.py`, `Balance_page.py`; implements core ATM operations
- **Data Layer** — `account.py` (in-memory account data), transaction and fraud alert log files

Machine Learning Layer:

- `train_model.py` — offline training script; generates synthetic transaction data and trains the model
- `fraud_detector.py` — inference module; loads the trained model and scores live transactions

Data Flow:

- **User Input** → **Business Logic Module** → **Balance/Amount Validation** → **ML Fraud Check** → **Decision (Allow/Flag/Confirm)** → **Update Account Data** → **Display Result / Log Alert**
- Clear separation of concerns between UI, business logic, data, and ML components makes the system easy to extend

Design Diagrams

Use Case Diagram

- Actor: User (Account Holder)
- Use Cases: Login, Deposit Money, Withdraw Money, Check Balance, View Fraud Alerts, Exit System
- Include relationship: Deposit and Withdraw both include "Run Fraud Check"

Workflow Diagram

- Start → Login (validate PIN, max 3 attempts) → Main Menu Loop →
- Deposit → Validate Amount → Fraud Check → Update Balance → Display Result
- Withdraw → Validate Balance → Fraud Check → Confirm if flagged → Update Balance → Display Result
- Check Balance → Display Summary
- View Alerts → Display Fraud Log
- → Exit → End

Sequence Diagram

User → main.py → respective module (e.g., withdraw_money.py) → account.py (balance check) → fraud_detector.py (ML check) → Response back to User

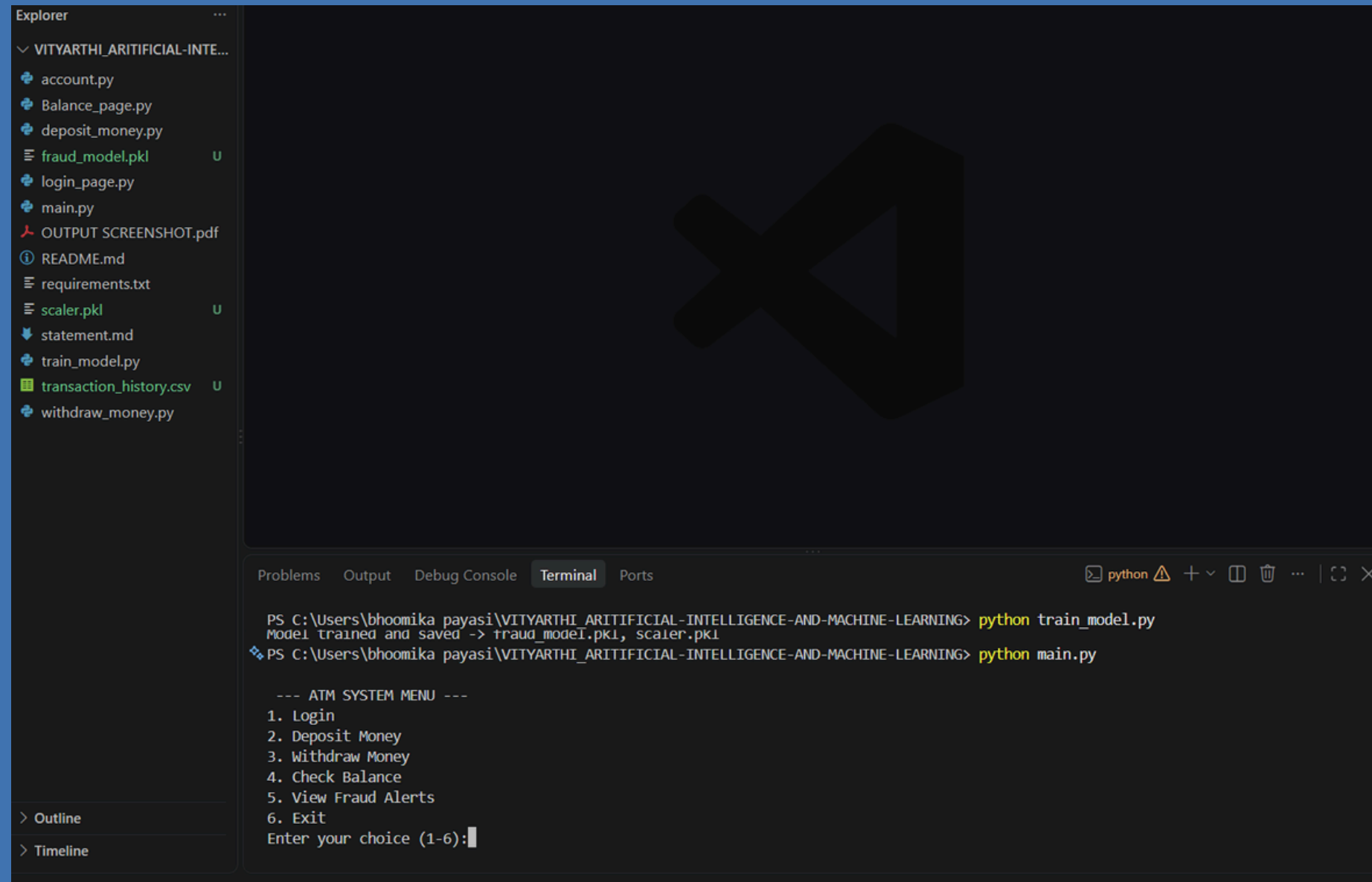
Design Decisions & Rationale

- **Modular file structure → easier debugging, testing, and scalability**
- **Isolation Forest chosen over supervised models → no labeled fraud dataset available, unsupervised suits real-world scarcity of confirmed fraud data**
- **Feature selection (amount, hour, day, balance ratio) → captures behavioral patterns without needing complex customer profiling**
- **Synthetic training data → allows model demonstration without real sensitive banking data**
- **Confirmation prompt on flagged withdrawals → balances security with user autonomy (avoids blocking legitimate transactions outright)**

Implementation Details

- **Language:** Python 3
- **Libraries:** scikit-learn (IsolationForest, StandardScaler), pandas, numpy, joblib
- **Model training:** train_model.py generates synthetic data, trains and saves model (fraud_model.pkl, scaler.pkl)
- **Inference:** fraud_detector.py loads saved model, scores each transaction using extracted features
- **Core Python concepts:** functions, loops, conditionals, file handling, module imports
- **Persistence:** trained model saved via joblib; fraud alerts logged to file

Screenshots / Results



```
--- ATM SYSTEM MENU ---
1. Login
2. Deposit Money
3. Withdraw Money
4. Check Balance
5. View Fraud Alerts
6. Exit
Enter your choice (1-6):
```

```
PS C:\Users\bhoomika payasi\VITYARTHI_ARITIFICIAL-INTELLIGENCE-AND-MACHINE-LEARNING> python main.py
6. EXIT
Enter your choice (1-6):5
Traceback (most recent call last):
  File "C:\Users\bhoomika payasi\VITYARTHI_ARITIFICIAL-INTELLIGENCE-AND-MACHINE-LEARNING\main.py", line 35, in <module>
    main()
    ~~~~^
  File "C:\Users\bhoomika payasi\VITYARTHI_ARITIFICIAL-INTELLIGENCE-AND-MACHINE-LEARNING\main.py", line 26, in main
    from fraud_detector import show_alerts
ModuleNotFoundError: No module named 'fraud_detector'
PS C:\Users\bhoomika payasi\VITYARTHI_ARITIFICIAL-INTELLIGENCE-AND-MACHINE-LEARNING>
PS C:\Users\bhoomika payasi\VITYARTHI_ARITIFICIAL-INTELLIGENCE-AND-MACHINE-LEARNING> 2
```

Testing Approach

- Login testing: correct PIN, incorrect PIN (multiple attempts), lockout behavior
- Deposit testing: valid amount, zero/negative amount, unusually large amount (fraud flag)
- Withdrawal testing: sufficient balance, insufficient balance, invalid input, large withdrawal (fraud flag + confirmation)
- Balance check: verifies correct updated balance and formatting
- Fraud alert testing: confirms flagged transactions are logged and viewable
- Exit testing: clean program termination

Challenges Faced

- Structuring an unsupervised ML model without a real labeled fraud dataset
- Selecting meaningful features that reflect real transaction anomalies
- Integrating ML predictions into a synchronous, console-based interactive flow
- Balancing false positives (blocking legitimate large transactions) vs. false negatives
- Ensuring in-memory account data stayed consistent across modules

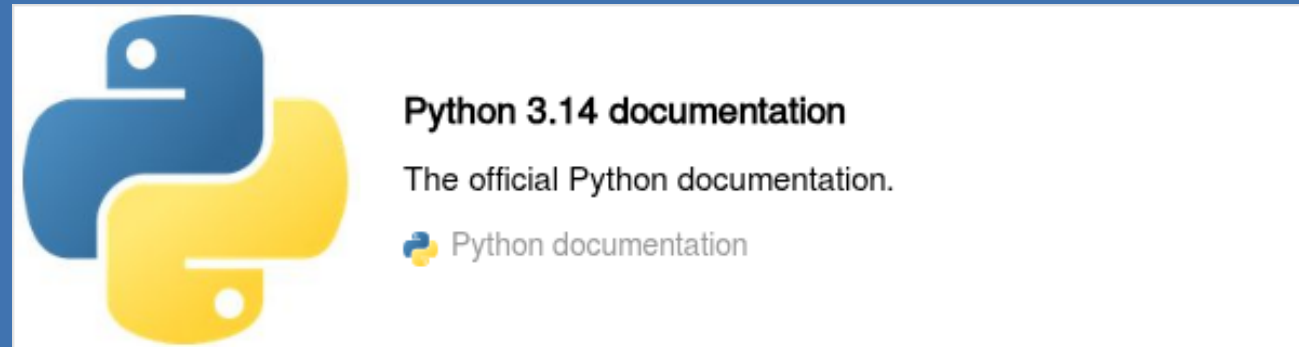
Learnings & Key Takeaways

- **Practical exposure to unsupervised anomaly detection (Isolation Forest)**
- **Understanding feature engineering for behavioral/transactional data**
- **Hands-on experience integrating ML models into a functional application**
- **Reinforced core Python skills: modular design, exception handling, file I/O**
- **Learned the real-world difference between rule-based systems and data-driven fraud detection**

Future Enhancements

- Train on real (anonymized) transaction datasets for improved accuracy
- Add supervised classification model if labeled fraud data becomes available
- Persistent database storage instead of in-memory account data
- Multi-user account support with authentication
- GUI or mobile front-end integration
- Real-time alerting system (e.g., SMS/email on flagged transactions)

References



Isolation Forest research paper: Liu, F.T., Ting, K.M., Zhou, Z.H. (2008). "Isolation Forest." IEEE ICDM

[scikit-learn documentation](#)

